

COMPUTER VISION

Unit 1: Image Formation & Fundamentals

Unit Overview

Unit No.	Topic Name	Sub-Topics Covered	Hours	CO Code
Unit 1	Image Formation & Fundamentals	Intro to CV & Visual Learning, Human vs Machine Vision; Image formation & pinhole camera model; Sampling & quantization; Colour models & image representation; Basic image processing (filtering & enhancement)	09	CO1

Introduction to Computer Vision and Visual Learning

What is Computer Vision?

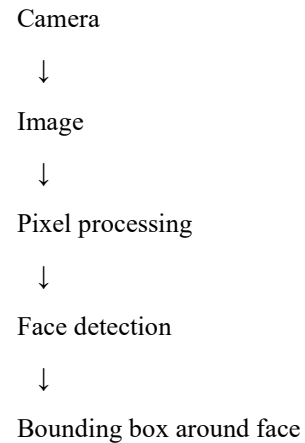
Computer Vision (CV) is the field of study that enables computers to derive meaningful, high-level understanding useful information from digital images or videos, in the same way that biological vision allows humans and animals to interpret their surroundings.

In simple terms:

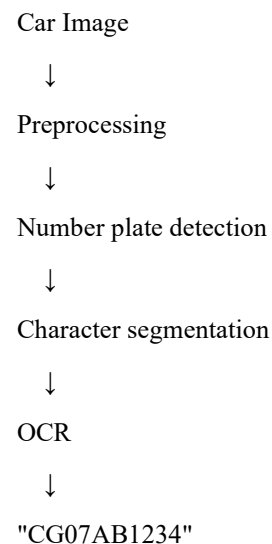
Image → Computer processes pixels → Extracts information → Makes a decision

Formally, Computer Vision can be defined as the automatic extraction, analysis, and understanding of useful information from a single image or a sequence of images.

For example: **Face detection**



Number plate recognition



What is an image?

An image is essentially a 2D array of numerical values.

For example, a grayscale image might look like:

```
10 20 30 40
15 25 35 45
20 30 40 50
25 35 45 55
```

Each number represents the intensity/brightness of one location.

- *So, Image is collection of pixels where each pixel contains numerical information.*
 - *A computer works with these numbers.*
 - *A 1920×1080 image contains:*

$$1920 \times 1080 = 2,073,600 \text{ pixels.}$$
So approximately 2 megapixels.
 - *Pixel = Picture Element*
 - *A pixel is the smallest individual element of a digital image.*
-

Computer Vision vs Image Processing

These two concepts are closely related but are **not the same**.

Image Processing

The goal is generally to **modify or improve an image**.

Examples:

- Resize image
- Remove noise
- Increase brightness
- Sharpen image
- Convert RGB $\rightarrow$ grayscale
- Blur image
- Enhance contrast

Input Image



Image Processing



Improved Image

Computer Vision

The goal is to **understand what is present in an image**.

Examples:

- Is there a person?
- Is this a cat or dog?
- Where is the car?
- How many people are present?
- Is the person wearing a helmet?

Input Image → Computer Vision → Understanding / Decision

Important relationship

Image Processing → Preprocessing → Computer Vision → Prediction / Understanding

So, image processing is often a **preprocessing stage of a CV system**.

What is Visual Learning?

Visual Learning refers to the process by which a system (biological or artificial) acquires the ability to interpret visual data through experience, i.e., by learning statistical regularities and patterns from a large number of example images rather than being explicitly programmed with hand-crafted rules.

In simple words, Visual learning refers to machine learning/deep learning systems that learn patterns from visual data.

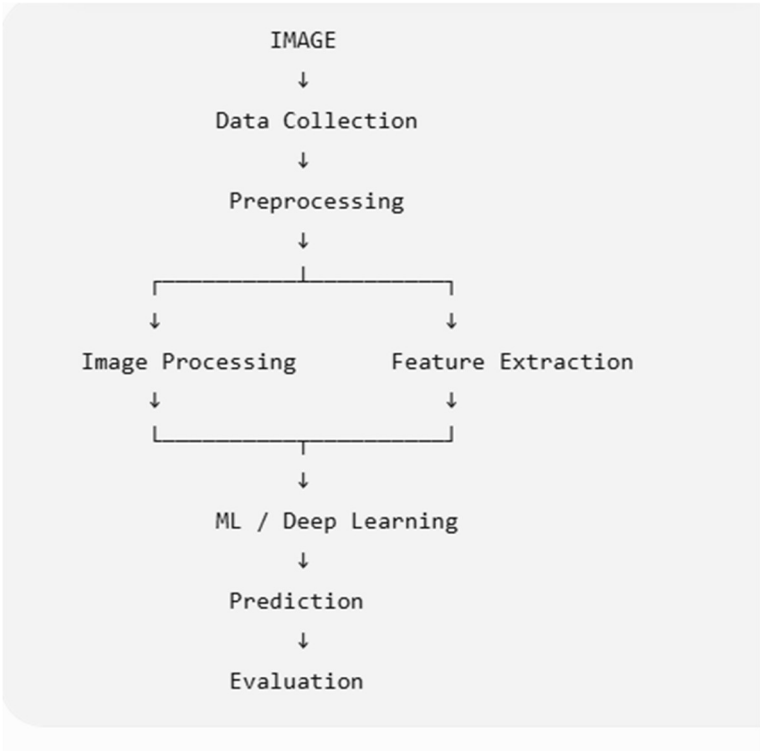
Examples:

Task	Input	Output
Image Classification	Image	Cat
Object Detection	Image	Cat + location
Image Segmentation	Image	Pixel-level regions
Face Recognition	Face image	Person identity
OCR	Text image	Text

Goals / Applications of Computer Vision

Application Area	Typical Tasks	Examples
Object Recognition	Detection, classification, localization	Face recognition, pedestrian detection
Scene Understanding	Segmentation, depth estimation, 3-D reconstruction	Autonomous driving perception stack
Motion Analysis	Optical flow, tracking, action recognition	Sports analytics, surveillance
Medical Imaging	Segmentation, anomaly detection	Tumor detection in MRI/CT scans
Industrial	Quality inspection, robotics guidance	PCB defect detection, bin picking
Biometrics	Iris/fingerprint/face recognition	Airport security, phone unlock
AR/VR	Pose estimation, SLAM	Markerless AR overlays

A practical CV pipeline often looks like:



Human Vision vs Machine Vision

Human Vision System

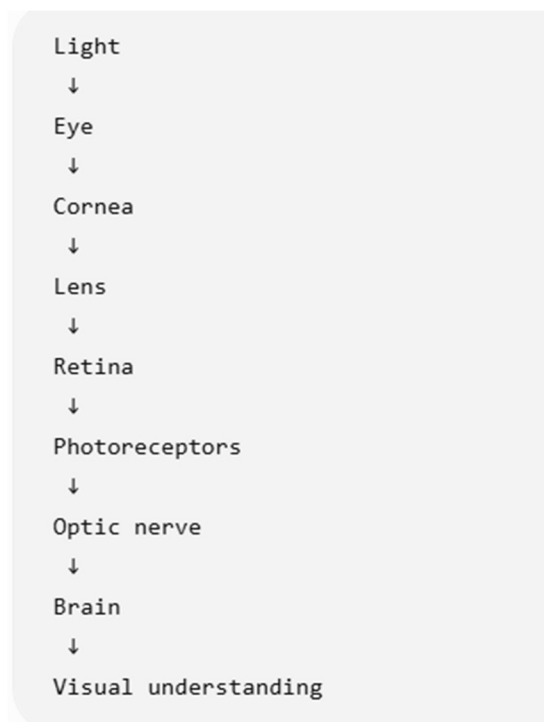
The human visual system begins with light entering the eye through the cornea and lens, which focuses an image onto the retina.

The retina contains two types of photoreceptors: rods (about 120 million, responsible for vision in low light and motion detection, but not color) and cones (about 6 million, responsible for color vision and fine detail, concentrated in the fovea).

Photoreceptor signals are pre-processed by retinal ganglion cells and transmitted via the optic nerve to the Lateral Geniculate Nucleus (LGN) and then to the primary visual cortex (V1) in the occipital lobe, where edge and orientation detection occurs.

Higher visual areas (V2, V4, IT cortex) progressively extract more complex features, ultimately enabling object recognition in the inferotemporal cortex.

Simplified Process



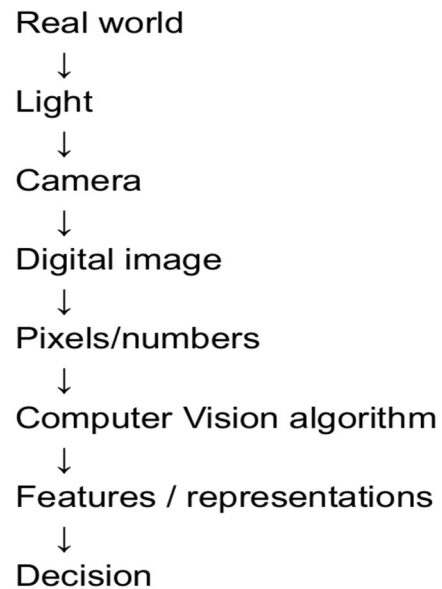
The retina contains:

- **Rods** → low-light vision
- **Cones** → color vision

The brain then interprets the information.

Machine Vision

A computer vision system uses:



Seeing a picture in which a red color car is parked.

A human may immediately understand: "There is a red car parked beside a road."

A computer initially sees something like:

Pixel(0,0) = [123, 80, 54]

Pixel(0,1) = [125, 82, 55]

Pixel(0,2) = [130, 85, 60]

...

It doesn't inherently know that these pixels represent a car.

A CV model must **learn visual patterns**.

Human Vision vs Machine (Computer) Vision

Human Vision

Eyes capture light

Retina processes signals

Brain interprets

Very strong contextual understanding

Learns naturally

Highly adaptable

Excellent generalization

Machine Vision

Camera captures light

Sensor produces digital values

Algorithms/models interpret

Context can be difficult

Requires data/training

Can fail under distribution changes

Usually task-dependent

Levels of Processing in a Computer Vision System (Marr's Paradigm)

David Marr proposed a very influential three-level framework for understanding vision, which is often used to structure CV pipelines:

1. Low-level vision: pixel-level operations such as edge detection, filtering, noise removal, and computation of the 'primal sketch' (basic image features like edges and blobs).
2. Mid-level vision: grouping and organization of low-level features into the '2.5-D sketch' — surfaces, depth, and orientation relative to the viewer, using cues like stereo, shading and texture.
3. High-level vision: object recognition, scene interpretation and construction of a full 3-D model of the scene, independent of viewpoint.

Some Important Questions & Answers

1. Define Computer Vision. How does it differ from Image Processing?
 - Image Processing focuses on transforming an image into another image (image-to-image, e.g., enhancement, denoising) whereas Computer Vision focuses on extracting meaning/understanding from an image (image-to-information, e.g., recognizing an object). Image processing is often a pre-processing stage used within a larger CV pipeline.
2. Differentiate between rods and cones in the human eye and explain their role in vision.
 - Rods (~120 million) are highly sensitive to light, enable scotopic (low-light) and peripheral vision, but do not detect color.
 - Cones (~6 million) are concentrated in the fovea, function in bright light (photopic vision), and provide color vision and high spatial acuity via three types sensitive to short, medium and long wavelengths.

Practical / Lab Exercise

Aim

To install and set up a Python-based Computer Vision environment (OpenCV, NumPy, Matplotlib), read a digital image, display its basic properties, and view it — as an introductory exercise to appreciate how a 'scene' becomes a numeric array a machine can process.

Requirements

- Python 3.8+, pip
- Libraries: opencv-python, numpy, matplotlib

Setup

```
pip install opencv-python numpy matplotlib
```

Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read an image from disk (BGR order by default in OpenCV)
img = cv2.imread('sample.jpg')

if img is None:
    raise FileNotFoundError('Check the image path')

# Basic properties -> demonstrates that an image is just a numeric array
print('Image shape (H, W, Channels):', img.shape)
print('Data type:', img.dtype)
print('Total pixels:', img.shape[0] * img.shape[1])
print('Pixel value at (0,0):', img[0, 0])

# Convert BGR (OpenCV default) to RGB for correct display with matplotlib
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb)
plt.title('Loaded Image')
plt.axis('off')
plt.show()
```

Expected Output

The console prints the image's shape (e.g., (408, 612, 3) meaning 408 rows, 612 columns, 3 colour channels), its data type (uint8), the total pixel count: 249696, and the BGR value of the top-left pixel. The Matplotlib window displays the loaded photograph in correct colours.

Image Formation Process and the Pinhole Camera Model

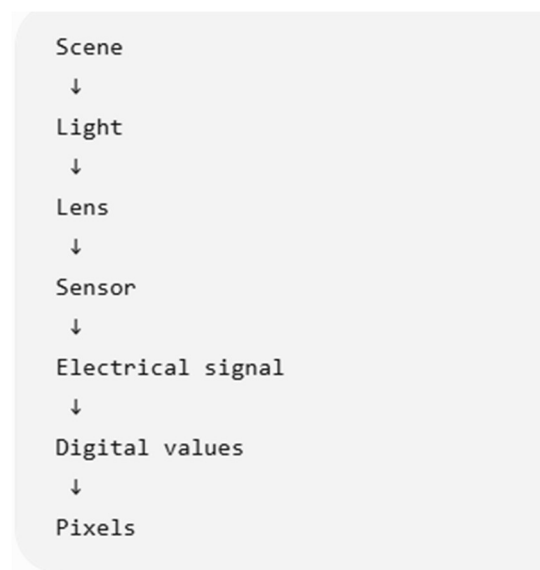
The Image Formation Process

Image formation is the physical process by which

Light reflected or emitted from objects in a 3-D scene enters the camera lens

The lens focuses the light onto the image sensor.

The sensor converts the light into electrical signals, which are then converted into digital values
(a 2-D array of intensity/colour values (an image)).



Three main factors govern image formation:

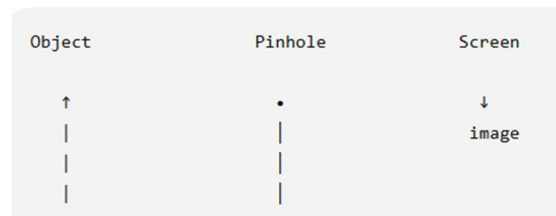
- Geometry: how 3-D points project onto the 2-D image plane (perspective projection), determined by camera position, orientation and optics.
- Radiometry (Photometry): how much light reaches the sensor from a given scene point, determined by illumination, surface reflectance (BRDF), and sensor sensitivity.
- Sensing: how continuous light intensity is converted into discrete digital numbers by the camera sensor (sampling + quantization + noise).

The Pinhole Camera Model

- The pinhole camera is the simplest and most fundamental model of image formation.
- It consists of a light-tight box with a very small aperture (the 'pinhole') on one side, through which light rays from the scene pass and form an inverted image on the opposite side (the image plane).
- Because the aperture is infinitesimally small, only a single ray of light from each scene point can pass through it, resulting in a sharp, focused image without the need for a lens (in the idealized model).

Imagine a completely dark box.

Put a tiny hole on one side.



Light rays from the object pass through the tiny hole and create an image on the opposite side.

The resulting image is inverted.

Geometry of Perspective Projection

Let the pinhole (also called the centre of projection or optical centre) be located at the origin O , and let the optical axis be the z -axis. The image plane is placed at a distance f (the focal length) from the pinhole, along the optical axis.

Consider a 3-D scene point $P = (X, Y, Z)$ expressed in the camera coordinate system.

$$x = f \frac{X}{Z}$$

$$y = f \frac{Y}{Z}$$

The important concept is:

Objects farther away appear smaller.

This relationship is called the perspective projection equation.

Homogeneous Coordinates and the Projection Matrix

In Computer Vision, we commonly write the projection as:

$$s \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = K \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

where K is the **camera intrinsic matrix**.

Typically:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

You don't need to deeply memorize this initially.

Understand the idea:

A camera converts 3D world coordinates into 2D image coordinates.

Practical / Lab Exercise

Aim

To simulate perspective projection of 3-D points using the pinhole camera model in Python, and to perform basic camera calibration to estimate intrinsic parameters and correct lens distortion using OpenCV.

Simulating Perspective Projection

```
import numpy as np

# Camera intrinsic matrix (fx, fy, cx, cy in pixels)
K = np.array([[800, 0, 320],
              [0, 800, 240],
              [0, 0, 1]], dtype=float)

# A set of 3-D points in the camera coordinate frame (X, Y, Z) in metres
points_3d = np.array([
    [0.30, 0.15, 3.0],
    [-0.20, 0.10, 2.0],
    [0.00, 0.00, 1.0],
])

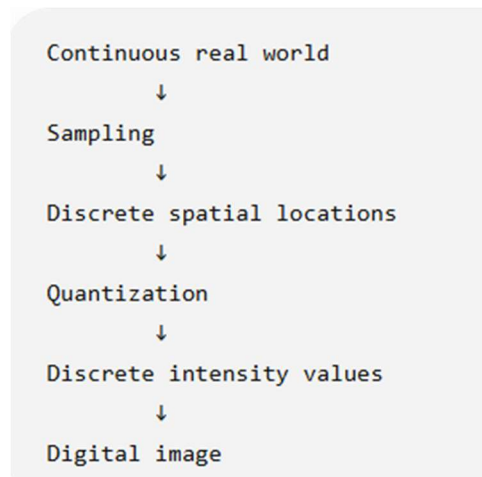
def project(points_3d, K):
    # Convert to homogeneous image coords: s*[x,y,1]^T = K * [X,Y,Z]^T
    proj = (K @ points_3d.T).T
```

```
# Perspective divide by Z (the third component before normalisation)
pixels = proj[:, :2] / points_3d[:, 2:3]
return pixels

pixel_coords = project(points_3d, K)
print('Projected pixel coordinates:\n', pixel_coords)
```

Sampling and Quantization

- A real-world scene has
 - continuous spatial variation (i.e. domain of the image is continuous $(x, y \in \mathbb{R})$ and
 - continuous intensity variation — light intensity is a continuous function $f(x, y)$ of continuous spatial coordinates.
- Light varies infinitely across space and brightness transitions smoothly from deep shadows to light shades and to store and process this on a digital computer, the continuous signal must be converted into a discrete, finite representation through two independent operations:
- sampling (discretizing the spatial coordinates) and
- quantization (discretizing the intensity/amplitude values)



Sampling

- Sampling = deciding WHERE we measure the image.
- Sampling is the process of measuring the value of the continuous image function $f(x, y)$ at a discrete, evenly spaced grid of spatial locations, producing an $M \times N$ array of samples (pixels).
- In an image, sampling determines the **spatial resolution**.
- For example:
 - 500×500
 - 1920×1080
 - 3840×2160
- More samples generally mean higher spatial detail.
- If sampling is too low, important details can disappear.
- The spacing between samples determines the spatial resolution of the digital image:
 - A smaller sampling interval (higher sampling rate) captures finer spatial detail, while
 - A larger interval loses detail and may introduce aliasing.
- **Sampling Interval** is physical distance between the centres of two adjacent sensor pixels (often called **pixel pitch**).
- **Sampling Rate** (f_s): The reciprocal of the interval. It measures how many samples (pixels) are collected per unit of physical distance (e.g., pixels per millimeter or DPI).

possible intensity values.

Examples:

0 → black
 50 → dark gray
 128 → medium gray
 200 → light gray
 255 → white

4-bit Image

If we use only 4 bits:

$$2^4 = 16$$

levels.

So instead of 256 intensity levels: 0, 1, 2, ..., 255

we have: 0, 1, 2, ..., 15

This produces less intensity precision.

Sampling vs Quantization

Think:

Sampling

↓

How many pixels?

Quantization

↓

How many brightness levels per pixel?

Image Representation as a Matrix

After sampling and quantization, a digital image is completely represented as a 2-D matrix

For grayscale: $f(x, y)$, $x = 0 \dots M-1$, $y = 0 \dots N-1$, where each element is an integer intensity value in $[0, L-1]$.

A colour image is represented as a 3-D array ($M \times N \times 3$) with separate matrices for each colour channel (commonly Red, Green, Blue).

Program: Effect of Quantization (Bit-depth Reduction)

```
import cv2, numpy as np
import matplotlib.pyplot as plt

img = cv2.cvtColor(cv2.imread('sample.jpg'), cv2.COLOR_BGR2GRAY)

def quantize(img, levels):
    step = 256 // levels
    return (img // step) * step

bit_levels = [256, 16, 8, 4, 2] # corresponds to 8,4,3,2,1 bits
fig, axes = plt.subplots(1, len(bit_levels), figsize=(15, 4))

for ax, L in zip(axes, bit_levels):
    q = quantize(img, L)
    ax.imshow(q, cmap='gray', vmin=0, vmax=255)
    ax.set_title(f'{L} levels')
    ax.axis('off')

plt.tight_layout()
plt.show()
```

Color Models and Image Representation

Now we need to understand how a computer represents color.

The most important models for practical CV work are:

1. Grayscale
2. RGB
3. HSV
4. HSL
5. CMYK

For computer vision, **RGB and HSV** are particularly important.

1.) Grayscale

A grayscale image has only one channel (i.e one value per pixel.)

A grayscale image contains: height * weight (example 640 * 480 values).

Example: Pixel = 90

Each pixel contains one intensity value:

0 → black

255 → white

- Grayscale requires much less storage.
-

Color image

A color pixel usually needs multiple values.

For RGB: Pixel = [R, G, B]

For example: [255, 0, 0] represents red in the standard RGB convention.

[0, 255, 0] green.

[0, 0, 255] blue.

2.) RGB Color Model of Image

RGB stands for:

- Red
- Green
- Blue

- RGB is an additive colour model in which Red, Green and Blue light are combined in varying intensities to produce a wide gamut of colours, matching the three types of cone cells in the human retina. It is represented geometrically as a unit cube with Black at the origin (0,0,0) and White at the opposite corner (1,1,1), with the greyscale (achromatic) axis running along the cube's main diagonal.
 - Used widely in displays and digital images
-

Example: Pixel = [R, G, B]

- For an 8-bit image:
- $R = 0-255$
- $G = 0-255$
- $B = 0-255$

Example:

[255, 0, 0] → Red

[0, 255, 0] → Green

[0, 0, 255] → Blue

[255,255,255] → White

[0,0,0] → Black

RGB Image Shape

Suppose: `image.shape` returns: (720, 1280, 3)

This means:

Height = 720

Width = 1280

Channels = 3

Total values: $720 * 1280 * 3$

RGB color mixing

In additive RGB:

Red + Green = Yellow

Green + Blue = Cyan

Red + Blue = Magenta

Red + Green + Blue = White

3.) CMY / CMYK Color Model

CMY (Cyan, Magenta, Yellow) is a subtractive colour model used primarily in colour printing, where each ink absorbs (subtracts) a portion of the reflected white light.

CMY has three inks:

- **C = Cyan** → bluish-green
- **M = Magenta** → pinkish-purple
- **Y = Yellow**

These inks work by **absorbing RGB colors**.

For example:

Ink	Mainly absorbs
Cyan	Red
Magenta	Green
Yellow	Blue

So if we put **cyan ink** on white paper, it absorbs much of the red light and reflects blue + green → your eyes see **cyan**.

The simple conversion from RGB is

$$C = 1 - R,$$

$$M = 1 - G,$$

$$Y = 1 - B \text{ (normalized to [0,1])}.$$

In practice, an additional black (K) channel is added (making CMYK) because combining C+M+Y inks produces a muddy dark brown rather than a true rich black, and using pure black ink for text/shadows is cheaper and sharper than mixing three coloured inks.

4.) HSV / HSI Color Model

HSV means :

- H → Hue
- S → Saturation
- V → Value

It is often easier for humans to reason about color using HSV.

and the closely related HSI (Hue, Saturation, Intensity)

Hue represents the basic color.

Red
Green
Blue
Yellow
...

Saturation

Represents how strong/pure the color is.

High saturation → vivid color
 Low saturation → washed-out/grayish

Value

Represents brightness.

High V → bright
 Low V → dark

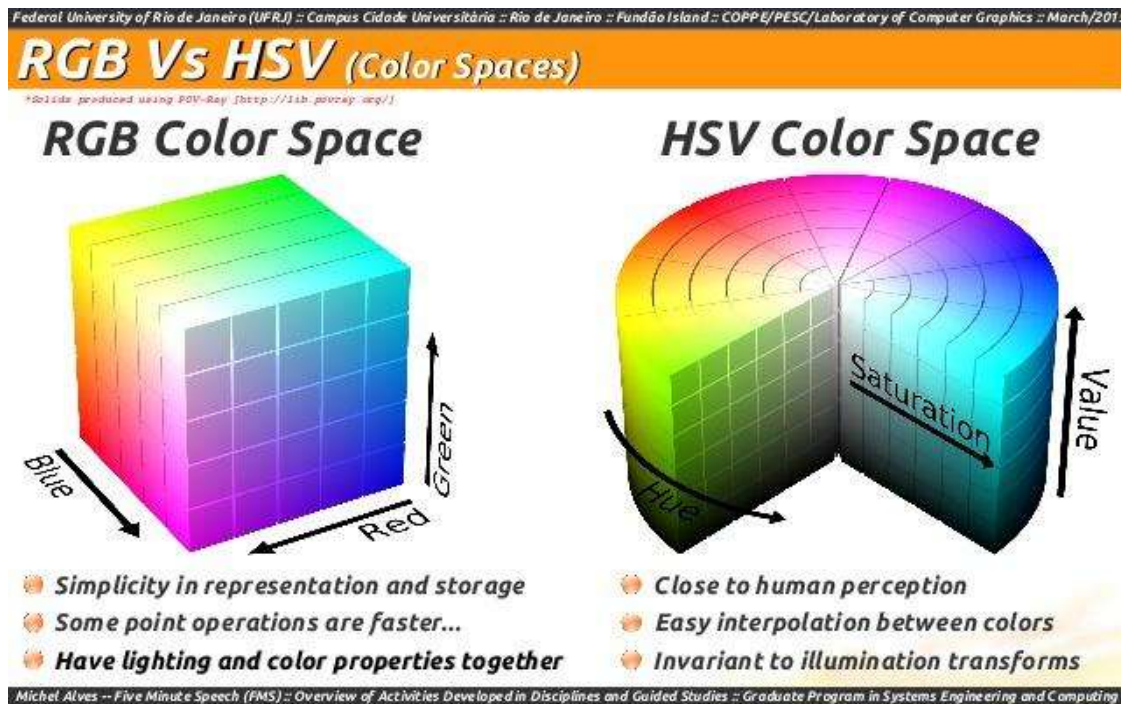
This separation is extremely useful in Computer Vision for tasks like colour-based object segmentation (e.g., detecting a red ball) because it makes the algorithm more robust to lighting/shadow changes, which mostly affect the V/I channel rather than H and S.

$$V = \max(R, G, B)$$

$$S = (V - \min(R, G, B)) / V \quad (\text{if } V \neq 0, \text{ else } S = 0)$$

Hue (H) is computed piecewise depending on which of R, G, B is maximum, mapping the colour to an angular position on the hue 'colour wheel' (0° = red, 120° = green, 240° = blue).

Why HSV is useful in Computer Vision



Suppose you want to detect **red** objects.

In RGB, lighting changes can make this difficult.

HSV lets you focus primarily on:

- Hue → Is it red?
- Saturation → Is the color strong enough?
- Value → Is it bright enough?

This is extremely useful for:

- Traffic light detection
 - Fruit detection
 - Color-based object tracking
 - Industrial inspection
-

Image Representation:

- Now understand how an image actually lives inside a computer.
- Digital images are stored as multi-dimensional numeric arrays.

Grayscale Image representation

A grayscale image of size M x N is a single 2-D array (M rows x N columns) of intensity values(brightness).

Suppose: Image size = 3 x 4

So matrix representation of above image can be like this:

```
10 20 30 40
50 60 70 80
90 100 110 120
```

Color representation

- A colour image is typically stored as an M x N x 3 array (three stacked 2-D channel planes).
 - An RGB image can be represented as:
 - Height x Width x Channels
 - For example: 480 x 640 x 3
 - Meaning: 480 rows 640 columns 3 color channels
-

Note:

- `Image = cv2.imread("cat.jpg")` to load a image
- `print(image.shape)` to check the size of image
- `print(image.dtype)` to check the datatype of the image after loading
- `cv2.imshow("cat.jpg")` to display a image after loading
- `pixel = image[100, 200]` to access a specific pixel

Questions & Numerical Problems

- Numerical: Convert the RGB colour (200, 50, 80) [0-255 range] to CMY.

Solution: Normalize: $R=200/255=0.784$, $G=50/255=0.196$, $B=80/255=0.314$.

$C = 1 - 0.784 = 0.216$,

$M = 1 - 0.196 = 0.804$,

$Y = 1 - 0.314 = 0.686$.

- Numerical: For the RGB colour (0, 128, 255), compute the approximate HSV Value (V) and Saturation (S).

Solution (0-255 scale):

$V = \max(0, 128, 255) = 255$.

$\min = 0$.

$S = (V - \min) / V = (255 - 0) / 255 = 1.0$ (fully saturated).

- Explain why colour-based object segmentation is usually performed in the HSV space rather than directly in RGB.
- Numerical: For an RGB pixel (100, 150, 200), compute the luminance Y using the standard formula

$Y = 0.299R + 0.587G + 0.114B$.

Solution: $Y = 0.299(100) + 0.587(150) + 0.114(200) = 29.9 + 88.05 + 22.8 = 140.75 \approx 141$.

Practical

Part 1: Color Space Conversions and Channel Visualization

```
import cv2
import matplotlib.pyplot as plt

img_bgr = cv2.imread('sample.jpg')
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
img_ycc = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2YCrCb)
img_lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)
img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

titles = ['RGB', 'HSV', 'YCbCr', 'Lab', 'Gray']
images = [img_rgb, img_hsv, img_ycc, img_lab, img_gray]

fig, axes = plt.subplots(1, 5, figsize=(18, 4))
for ax, title, im in zip(axes, titles, images):
    cmap = 'gray' if title == 'Gray' else None
    ax.imshow(im, cmap=cmap)
    ax.set_title(title)
    ax.axis('off')
plt.tight_layout()
plt.show()

# Visualize individual HSV channels separately
H, S, V = cv2.split(img_hsv)
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
for ax, ch, name in zip(axes, [H, S, V], ['Hue', 'Saturation', 'Value']):
    ax.imshow(ch, cmap='gray')
    ax.set_title(name)
    ax.axis('off')
plt.show()
```

Part 2: HSV-based Colour Segmentation (e.g., isolating red objects)

```
import cv2, numpy as np

img = cv2.imread('sample.jpg')
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Red wraps around the hue circle, so two ranges are needed
lower_red1 = np.array([0, 120, 70]); upper_red1 = np.array([10, 255, 255])
lower_red2 = np.array([170, 120, 70]); upper_red2 = np.array([180, 255, 255])

mask1 = cv2.inRange(hsv, lower_red1, upper_red1)
mask2 = cv2.inRange(hsv, lower_red2, upper_red2)
mask = cv2.bitwise_or(mask1, mask2)

result = cv2.bitwise_and(img, img, mask=mask)

cv2.imwrite('red_mask.jpg', mask)
cv2.imwrite('red_segmented.jpg', result)
```

Basic Image Processing (Filtering and Enhancement)

Image processing means applying mathematical operations to an image to:

- improve its quality
- remove unwanted noise
- enhance important information
- extract useful details
- prepare the image for further computer vision tasks



Common operations:

Image Resizing:

To maintain the dimension as if aspect ratio is not preserved, objects may become distorted.

```
resized = cv2.resize(image, (224, 224))
```

Cropping means selecting a region of interest to remove irrelevant background

```
crop = image[100:400, 200:600]
```

Image Rotation

```
rotated = cv2.rotate(  
    image,  
    cv2.ROTATE_90_CLOCKWISE  
)
```

Grayscale Conversion

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

Why convert to grayscale?

Because many operations only need intensity information.

Examples:

- Edge detection
 - Thresholding
 - Contour detection
 - OCR preprocessing
-

1. What is Image Filtering?

Image filtering refers to modifying or enhancing an image by applying a mathematical operation, typically involving a small matrix called a kernel (or mask/filter), which is systematically applied across the image — most commonly through convolution or correlation.

Filtering is used for

- noise removal,(to get cleaner images)
- smoothing,
- sharpening,(to get more prominent details)
- edge detection,

and many other pre-processing tasks that improve the visual quality or highlight useful features of an image.

Kernel / filter mask

A **kernel**, also called a **filter mask**, is a small matrix used to process an image.

Example:

$$K = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

This kernel is moved across the image.

Mean/Average Filter

Suppose we have:

10	20	30
20	40	20
30	20	10

- In this filtering technique we replace the centre pixel (here 40) by the average value of all pixels
- If we do this throughout the image, the image becomes smoother.
- One of the drawbacks of mean filtering is that it can but can destroy important edges..

Gaussian Filtering

- A better smoothing technique is the **Gaussian filter**.
- In this technique, instead of giving every neighbouring pixel equal importance, it gives more weight to pixels near the centre..

1	2	1
2	4	2
1	2	1

Gaussian filter

1/16 ×		
1	2	1
2	4	2
1	2	1

Normalised Gaussian filter

```
blur = cv2.GaussianBlur(image, (5, 5), 0)
```

Median Filtering

Median filtering is extremely important.

It replaces the current pixel with the **median value** of its neighborhood.

Example:

10	11	10
12	255	11
10	12	10

The 255 is likely noise.

Sorted: 10 10 10 10 11 11 12 12 255

Median: 11

So the noisy 255 gets replaced by approximately 11.

```
filtered = cv2.medianBlur(image, 5)
```

- Median filtering is particularly effective against: **Salt-and-pepper noise**
- **Salt-and-pepper noise** is a type of image noise where some pixels are randomly changed to **very dark (black)** or **very bright (white)** values.
- It is called *salt-and-pepper* because the noisy image looks like someone sprinkled:
 - **Salt** → **white pixels**
 - **Pepper** → **black pixels**

Mean vs Gaussian vs Median

Filter	Main idea	Typical use
Mean	Average neighbours	Basic smoothing
Gaussian	Weighted average	Noise reduction/smoothing
Median	Middle value	Salt-and-pepper noise

Image Enhancement Techniques (Point Processing)

Image enhancement attempts to improve the visual quality or make important structures easier to analyze.

Examples:

- Brightness adjustment
 - Contrast enhancement
 - Histogram equalization
 - CLAHE
 - Sharpening
-

Brightness and Contrast Adjustment

- Brightness can be changed by adding a constant.
- Example: $\text{New Pixel} = \text{Old Pixel} + \beta$
- Increase the value of β and decrease its value to make a image dark.

- Contrast represents the difference between dark and bright regions.

A simple transformation is:

$$I_{new} = \alpha I_{old}$$

where:

- $\alpha > 1 \rightarrow$ increase contrast
- $\alpha < 1 \rightarrow$ decrease contrast

A basic linear point-processing transformation adjusts each pixel independently:

$$g(x, y) = \alpha \cdot f(x, y) + \beta$$

where

$f(x, y)$ = original image

α (alpha, $\alpha > 0$) controls contrast and β (beta) controls brightness

Values are typically clipped to the valid range $[0, 255]$ after the transform.

`adjusted = cv2.convertScaleAbs(image, alpha=1.5, beta=20)`

Histogram Equalization

A histogram shows: **How many pixels have each intensity value.**

For a grayscale image:

Intensity: 0 ----- 255 as its size is 1 byte

Example

Suppose an image contains:

For intensity value 10 $\rightarrow$ 100 pixels

For intensity value 20 $\rightarrow$ 200 pixels

For intensity value 30 $\rightarrow$ 500 pixels

For intensity value 40 $\rightarrow$ 300 pixels

The histogram helps us understand:

- whether an image is dark (most of our image pixel will have low value as the value for black is 0)
- whether it is bright (most of our image pixel will have high value as the value for white is 255)
- whether contrast is low (Pixel values occupy a small range)
- whether intensity values are spread out

Histogram equalization is a non-linear point-processing technique that redistributes pixel intensities so that the output histogram is approximately uniform, thereby improving the global contrast of images that are too dark, too bright, or have low dynamic range.

In simple words:

- Suppose an image has poor contrast: Most pixels lie in the range of 80–130. Instead of using the entire: 0–255 range.
- Histogram equalization attempts to spread intensity values over a wider range.

It works by computing the cumulative distribution function (CDF) of the input histogram and using a scaled version of the CDF as the intensity mapping function:

$$s = T(r) = (L-1) \cdot CDF(r)$$

where

- r is the input intensity,
- s is the output intensity, and
- L is the number of grey levels.

Adaptive variants like CLAHE (Contrast Limited Adaptive Histogram Equalization) apply this locally in small tiles with a clip limit to avoid over-amplifying noise in near-uniform regions.

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
equalized = cv2.equalizeHist(gray)
```

Questions & Numerical Problems

- Differentiate between linear and non-linear spatial filters, giving one example of each.
- Explain why a median filter is preferred over a mean filter for removing salt-and-pepper noise.
- Numerical: Apply a 3x3 mean filter to the pixel neighbourhood below and compute the filtered value of the centre pixel:

10	20	10
30	200	40
20	10	30

Solution:

Sum = $10+20+10+30+200+40+20+10+30 = 370$.

Mean = $370/9 \approx 41.1 \rightarrow$

the centre pixel (originally 200, an outlier/noise spike) is smoothed to about 41.

- For the same neighbourhood as above, compute the output of a 3x3 median filter at the centre pixel.

Solution: Sorted values: 10,10,10,20,20,30,30,40,200.

The median (5th of 9 values) = 20.

Notice the median filter almost completely removes the 200 outlier while the mean filter is still skewed by it — demonstrating the median filter's robustness to impulse noise.

- Numerical: Given the linear contrast transform $g = 1.5f - 20$, find the output intensity for input pixel values $f = 100$ and $f = 240$ (clip to $[0,255]$).

Solution: For $f=100$: $g = 1.5(100) - 20 = 130$.

For $f=240$: $g = 1.5(240) - 20 = 340 \rightarrow$ clipped to 255.
